

The F Prime Data Structure Library

Rob Bocchino
NASA Jet Propulsion Laboratory
February 11, 2026

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Introduction

- A **data structure** is
  - A collection of data
  - An interface and implementation for operating on the collection
- Using good data structures is key to writing code that
  - Is correct and performs well
  - Is understandable and maintainable
- F Prime provides a library of basic data structures
- This section of the course will
  - Review some basic data structures and their operations
  - Give an overview of the F Prime data structure library

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Basic Data Structures

- **Array:** A fixed-size map of numbers to elements
- **Queue:** Insertion and removal in corresponding order
- **Map:** A variable-size mapping from keys to values
- **Set:** Insertion and removal in any order

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Data Structures in FSW

- Many standard data structures allocate and free elements on demand
- In FSW we do not do that
- FSW data structures must
  - Use a fixed total size of memory to hold their data
  - Call *malloc* or *new* only at FSW startup
- Our data structures have **capacity** and **size**
  - **Capacity:** The max number of elements in the container (always fixed)
  - **Size:** The current number of elements (may vary)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

The F Prime Data Structure Library

- Located at *Fw/DataStructures*
- Written using C++ templates
- Provides basic data structures
- Provides **internal** and **external** variants of each data structure
  - **Internal:** Memory managed by the data structure instance
  - **External:** Memory managed by the code that instantiates the data structure
- All memory is allocated at startup and freed at exit

https://fprime.jpl.nasa.gov/latest/Fw/DataStructures/docs/sdd/

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Arrays

- Provide bounds-checked array indexing
- Array example

```
Array<U32, 10> a();
a[0] = 5;
```

- External array example

```
constexpr FwSizeType size = 10;
U32 elements[size];
ExternalArray<U32> a(elements, size);
a[0] = 5;
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Stacks

- Provide last in, first out insertion (push) and removal (pop)
- Stack example

```
Stack<U32, 10> stack;
auto status = stack.push(3);
ASSERT_EQ(status, Success::SUCCESS);
```

- External stack example

```
constexpr FwSizeType capacity = 10;
U32 items[capacity];
ExternalStack<U32> stack(items, capacity);
```

- *StackBase*: An abstract superclass of internal and external stacks

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

FIFO Queues

- Provide first in, first out insertion (enqueue) and removal (dequeue)
- FIFO queue example

```
FifoQueue<U32, 10> queue;
auto status = queue.enqueue(3);
ASSERT_EQ(status, Success::SUCCESS);
```

- External FIFO queue example

```
constexpr FwSizeType capacity = 10;
U32 items[capacity];
ExternalFifoQueue<U32> queue(items, capacity);
```

- *FifoQueueBase*: An abstract superclass of internal and external queues

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Maps

- Provide key-based insert, remove, and find
- There are two implementations
  - **ArrayMap:** Simple implementation for small containers
  - **RedBlackTreeMap:** Balanced binary tree; use for large containers
- ArrayMap example

```
ArrayMap<U16, U32, 10> map;
U16 key = 0;
U32 value = 42;
auto status = map.insert(key, value);
ASSERT_EQ(status, Success::SUCCESS);
```

- *MapBase*: An abstract superclass

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Sets

- Provide insert and remove
- There are two implementations
  - **ArraySet:** Simple implementation for small containers
  - **RedBlackTreeSet:** Balanced binary tree; use for large containers
- ArraySet example

```
ArraySet<U32, 10> set;
U32 element = 42;
auto status = set.insert(key, value);
ASSERT_EQ(status, Success::SUCCESS);
```

- *SetBase*: An abstract superclass

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Iterators

- An iterator is an object that
  - Points into a data structure being traversed
  - Stores the state of the traversal
- F Prime sets and maps provide C++ STL-style iterators
  - Call *begin()* to get an iterator that points to the start of the data structure
  - Call ++ to advance the pointer
  - Compare to *end()* to check whether the iterator is in range
- Special syntax for loops

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Iterator Example

```
void f(MapBase<U16, U32>& map) {
    map.clear();
    // Insert an entry in the map
    const auto status = map.insert(0, 1);
    ASSERT_EQ(status, Fw::Success::SUCCESS);
    // Get a map const iterator object
    auto it = map.begin();
    // Use the iterator to access the underlying map const entry
    const auto key = it->getKey();
    const auto value = it->getValue();
    ASSERT_EQ(key, 0);
    ASSERT_EQ(value, 1);
}
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.